

1. Student Average

Aim: To calculate the average marks of each student and identify the subject with the highest average.

Algorithm:

1. Start
2. Read students-data.csv
3. Extract subjects marks
4. Compute average of each subject
5. Find highest average subject.
6. Display results.

Code:

```
import numpy as np
```

```
student_scores = np.loadtxt (
```

```
    "students-data.csv",
```

```
    delimiter=",",
```

```
    skiprows=1,
```

```
    usecols=(2,3,4,5)
```

```
subjects = ["Math", "Science", "English", "History"]
```

```
average_scores = np.mean (student_scores, axis=0)
```

```
highest_subjects = subjects [np.argmax (average_scores)]
```

```
print ("Average scores:", np.round (average_scores, 2))
```

```
print ("Subjects with highest average score:", highest_sub)
```

Output:

Average scores: [81.25 83.5 84. 83.]

Subjects with highest average scores = English.

Result:

Thus, the average marks of each subject were calculated successfully and the highest average subject was identified.

2. past Month price Average

Aim: To calculate the average price for the past month using pandas.

Algorithm:

1. Start
2. Read sales-data.csv
3. Filter November 2023 records.
4. Calculate average price
5. Display result.
6. Stop

Code:

```
import pandas as pd
df = pd.read_csv('sales-data.csv')
average_price = df[df['Month-sales'] == 'November 2023']
                  ['price'].mean()
```

```
print("Average price:", round(average_price, 2))
```

Output:

Average price in November 2023 (past month): 165.0

Result:

Thus, the average price for the selected month was calculated successfully.

3. House Average price

Aim: To calculate the average price of houses having more than 4 bedrooms.

Algorithm:

1. Start
2. Read House-data.csv
3. Select houses with bedrooms > 4
4. Calculate average price
5. Display result
6. Stop.

Code:

```
import numpy as np
house_data = np.loadtxt("House data.csv",
                        delimiter=",",
                        skiprows=1,
                        usecols=(1,2,3,5))
house = house_data[house_data[:,0]>4]
average_price = np.mean(house[:,2])
print("Average price :", round(average_price, 2))
```

Output:

Average price of houses with more than 4 bedrooms
= 483333.33

Ok Ver

Result:

Thus, the average price of houses with more than four bedrooms was calculated successfully.

Handwritten signature

4. Quarterly Sales Analysis

Aim: To calculate the total yearly sales and percentage increase from Q1 to Q4

Algorithm:

1. Start
2. Read sales-data.csv
3. Extract month and sale columns
4. Calculate Q1 and Q4 sales
5. Calculate yearly sales
6. Display Result.

Code:

```
import numpy as np
Sales_data = np.genfromtxt("sales_data.csv",
                             delimiter=',',
                             skip_header=1,
                             dtype=str)
months = Sales_data[:,1]
Sales = Sales_data[:,4].astype(float)
Q1 = 0
Q4 = 0
for i in range(len(months)):
    if months[i] in ["January", "February", "March"]:
        Q1 += Sales[i]
    elif months[i] in ["October", "November", "December"]:
        Q4 += Sales[i]
total = np.sum(Sales)
increase = (Q4 - Q1) / Q1 * 100
print("Total sales: ", total)
print("percentage increases: ", round(increase, 2), "%")
```

Output: to generate the report with statistics of monthly sales and percentage increase in sales

8/12 ✓

Data read from CSV: transactional_data.csv

```
[['October 2023', '120']  
['November 2023', '150']  
['November 2023', '180']  
['December 2023', '200']
```

Shape (4,2)

Result:

Thus, the yearly sales and percentage increases from Q1 to Q4 were calculated successfully.

5 Fuel Efficiency Analysis

Aim: To calculate the average fuel efficiency of all cars, compare Mazda and Audi and find the percentage improvement.

Algorithm:

1. Start
2. Import Numpy
3. Read fuel_data.csv
4. Calculate average
5. Find percentage improvement
6. Display result
7. Stop

Code:

```
import numpy as np
fuel_data = np.genfromtxt("fuel_data.csv", delimiter=",",
                          skip_header=1, dtype=str)

fuel_efficiency = fuel_data[:, 7].astype(float)
make = fuel_data[:, 8]
average_efficiency = np.mean(fuel_efficiency)
mazda_avg = np.mean(fuel_efficiency[make == 'Mazda'])
audi_avg = np.mean(fuel_efficiency[make == 'Audi'])
percentage_improvement = ((mazda_avg - audi_avg) / audi_avg) * 100

print("Average fuel efficiency of all cars: ", round(average_efficiency, 2), "MPG")
print("Average fuel efficiency of Mazda: ", round(mazda_avg, 2), "MPG")
print("Average fuel efficiency of Audi: ", round(audi_avg, 2), "MPG")
print("percentage improvement: ", round((percentage_improvement, 2), 1), "%")
```

Output:

Average fuel efficiency of all cars: 27.25 mpg

Average fuel efficiency of Mazda: 28.80 mpg

Result:

Average fuel efficiency of Audi: 26.00 mpg

percentage Improvement : 9.62%

Result :

Thus the average fuel efficiency and percentage Improvement was calculated Successfully.

6. Customer Purchase

Aim: To calculate the total sale and final amount after applying discount and tax

Algorithm:

1. Start
2. Import pandas
3. Read groceries.csv
4. Calculate total sales
5. Apply discount
6. Apply tax
7. Display result
8. Stop.

Code:

Import pandas as pd

df = pd.read_csv("groceries.csv")

total_sales = df["sales"].sum()

discount = 0.10

tax = 0.18

price_after_discount = total_sales * (1 - discount)

price_after_tax = price_after_discount * (1 + tax)

print("Total sales: ", total_sales)

print("Final amount: ", round(price_after_tax, 2))

Output:

Total sales : 1500

Final amount : 1593.0

Result:

Thus, the total sales and final amount were calculated successfully.

7. Customer order Analysis

Aim: To analyze customer orders and determine order value and other dates.

Algorithm:

1. Start
2. Import pandas
3. Read order data.csv
4. Group customer orders
5. Calculate average order value
6. Find earliest and latest dates
7. Display results.

Code:

```
import pandas as pd

df = pd.read_csv("order data.csv")

total_orders = df.groupby("Full Name")["order count"].sum()

avg = df.groupby("Items")["order Total"].mean()

earliest = df["order data"].min()

latest = df["order data"].max()

print(total_orders)

print(avg)

print(earliest)

print(latest)
```

Output:

Alice	4
Bob	2
John	3
Milk	200.0
Oil	300.0
Rice	500.0
Sugar	150.0

2023-01-10

2023-08-22

Result:

Thus, the customer order details were analyzed successfully.

8. Five Most Sold Products

Aim: To identify the top 5 sold products based on the total sales amount using the sales_data.csv dataset.

Algorithm:

1. Import the pandas library.
2. Read the sales_data.csv file
3. Group the data by product_name
4. Calculate the total sales for each product
5. Display the top 5 product with the highest total sales.

Code:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
top-5-products = df.groupby("product_name")["price"].sum()
print("Five most sold products:")
print(top-5-products)
```

Output:

Five most sold products:

product_name	
Laptop	250000
Mobile	180000
Headphone	90000
Keyboard	60000
Mouse	50000

Result:

The top five products with the highest total sales were successfully displayed.

9. House Data Analysis

Aim: To analyze house data by finding the average house price for each location, the number of houses with more than four bedrooms, and the largest house size.

Algorithm:

1. Import the pandas library
2. Read the house.csv file
3. Group the data by location and calculate the average price
4. Count houses having more than four bedrooms.
5. Find the maximum house size.
6. Display all the results.

```
code:
import pandas as pd
df = pd.read_csv("house.csv")
location_price_average = df.groupby("location")["price"].mean()
more_than_4beds = df[df["beds"] > 4].count()["beds"]
largest_sq_feet = df["size"].max()
```

```
print("Number of houses with more than 4 bedrooms:",
      more_than_4beds)
print("largest house size:", largest_sq_feet)
```

Output:

Average house price for each location:

location

Delhi 6500000

Hyderabad 7200000

Mumbai 9800000

Number of houses with more than 4 bedrooms: 8

largest house size: 4500

Result: The average price by location with 4 bedrooms displayed successfully.

10. Monthly Sales Visualization

Aim: To visualize monthly sales data using line and bar charts.

Algorithm:

1. Import the pandas and matplotlib libraries.
2. Read the sales-data.csv file.
3. Group the data by month-sales.
4. Calculate the total size for each month.
5. Plot a line chart.

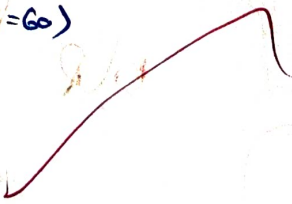
Code:

```
import pandas as pd
import matplotlib.pyplot as plt

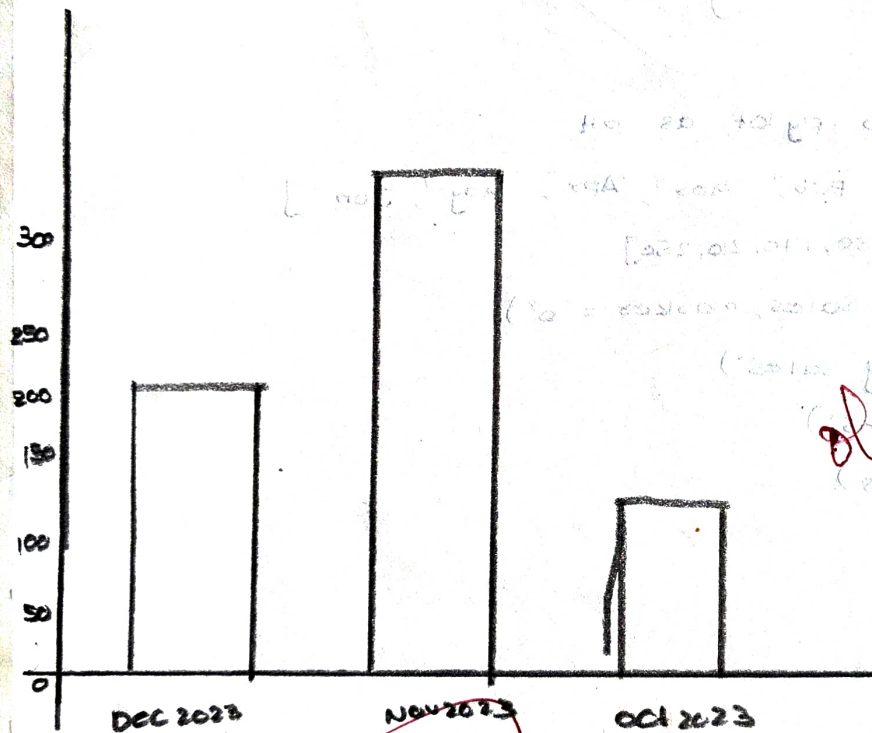
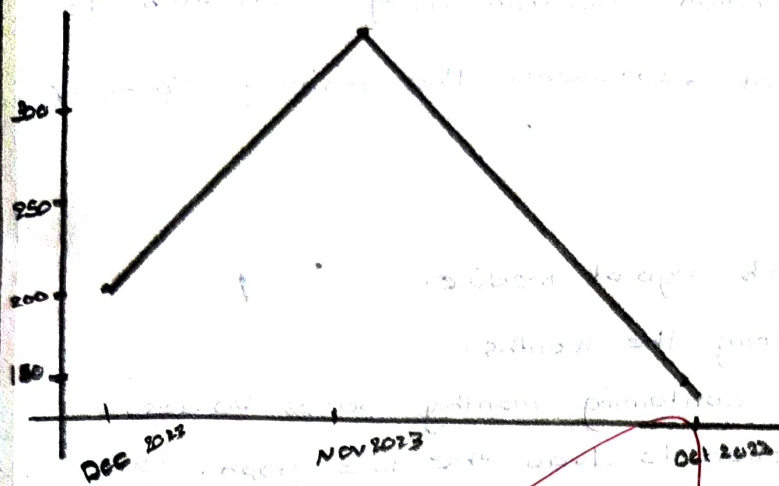
df = pd.read_csv("sales_data.csv")
monthly_sales = df.groupby("month-sales")["price"].sum()

plt.plot(monthly_sales.index, monthly_sales.values, markers='o')
plt.title("Monthly Sales Data")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation=60)
plt.show()

plt.bar(monthly_sales.index, monthly_sales.values)
plt.title("Monthly Sales Data")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation=60)
plt.show()
```



Output :



Result: The monthly sales data was successfully visualized using both line and bar charts, making it easy to compare sales across different months.

Handwritten signature
23/7/26

II. Line plot of Monthly Sales

Aim: To develop a python program using Matplotlib to create a line plot that represents the monthly sales of a product.

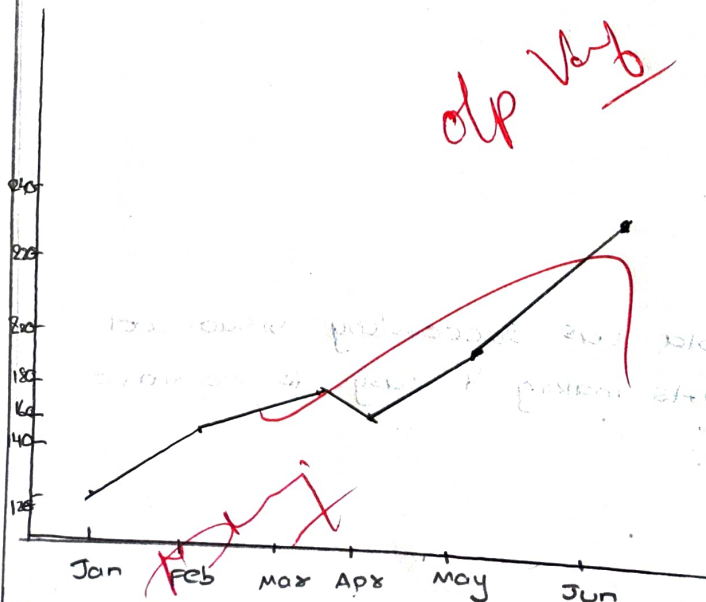
Algorithm:

1. Import the matplotlib.pyplot module.
2. Create a list containing the months.
3. Create another list containing monthly sales values.
4. Use the plot() function to draw the line graph.
5. Add title, x-axis label, and y-axis label.
6. Display the graph using show().

Program:

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
Sales = [120, 150, 180, 170, 210, 250]
plt.plot(months, Sales, marker='o')
plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

Output:



Result: Thus the python program to create a line plot for monthly.

11.2 Scatter plot of Monthly sales

Aim: To develop a python program using Matplotlib to create a scatter plot for monthly sales.

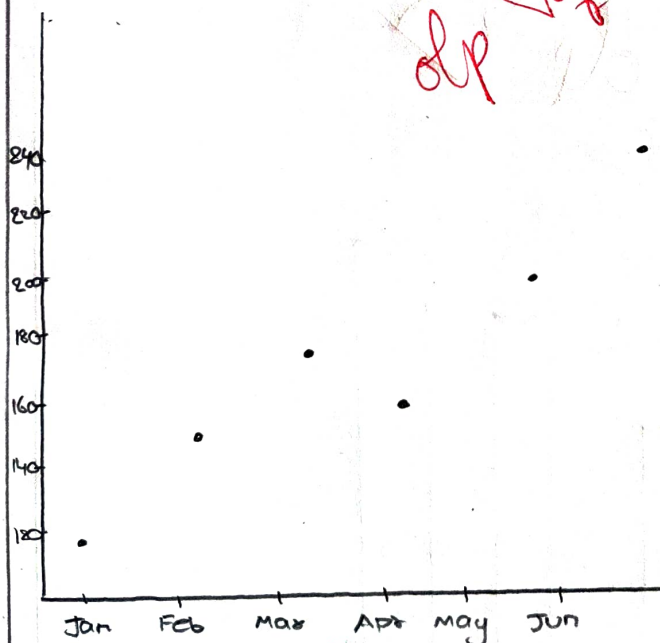
Algorithm:

1. Import the matplotlib library.
2. Store months and sales value in lists.
3. Use the scatter() functions.
4. Add title and axis labels.
5. Display the scatter plot.

Program:

```
import matplotlib.pyplot as plt
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
Sales = [120, 150, 180, 160, 210, 250]
plt.scatter(months, Sales)
plt.title("Monthly Sales Scatter Plot")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

Output:



Result:

Thus, the python program to create a scatter plot for monthly sales was executed successfully.

11.3 Bar plot of Monthly sales

Aim: To develop a python program to create a bar chart representing monthly sales.

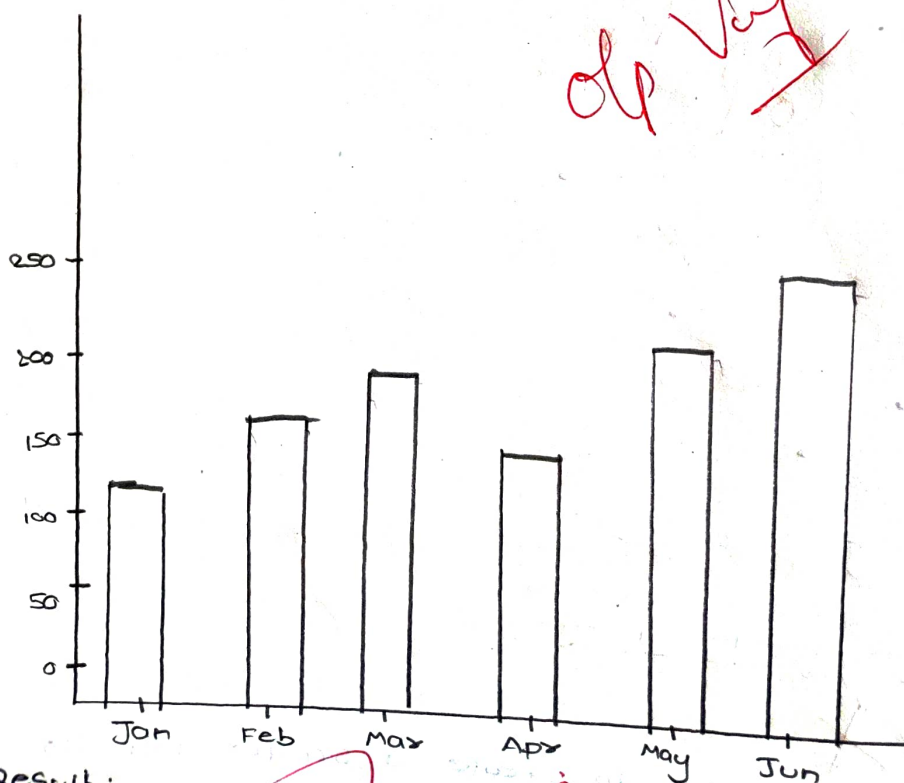
Algorithm:

1. Import matplotlib
2. Store months and sales values
3. Use the bar() function.
4. Add title and labels
5. Display the chart.

program:

```
import matplotlib.pyplot as plt
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [120, 150, 180, 140, 210, 250]
plt.bar(months, sales)
plt.title("Monthly sales Bar Chart")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```

Output:



Result:

Thus the python program to create a bar chart for monthly sales.

1 line plot of Monthly Temperature

Aim: To develop a python program to create a line plot showing monthly temperature variations.

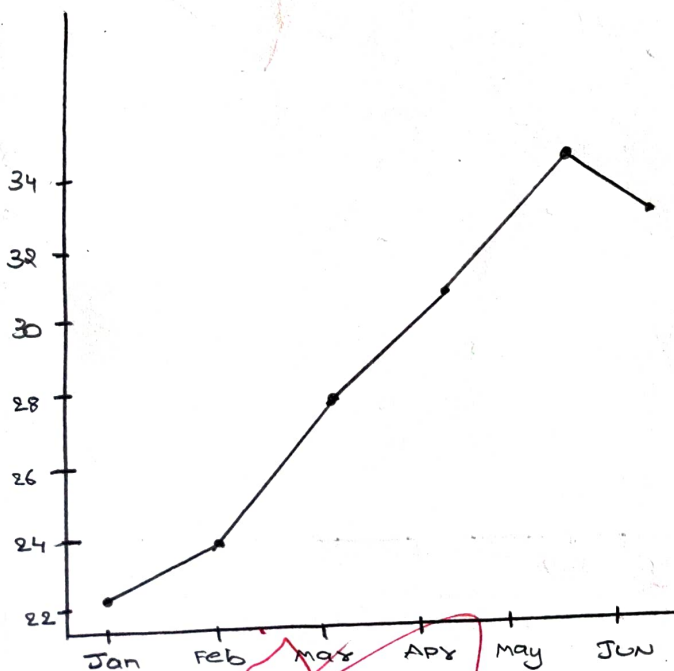
Algorithm:

1. Import matplotlib
2. Create lists for months and temperature values.
3. Use the plot() function
4. Add title and labels
5. Display the graph.

Program:

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
temperature = [22, 24, 28, 31, 35, 33]
plt.plot(months, temperature, marker='o')
plt.title("Monthly Temperature")
plt.xlabel("Months")
plt.ylabel("Temperature (°C)")
plt.grid(True)
plt.show()
```

Output:



Result:

Thus, the python program to create a line plot for monthly temperature was executed successfully.

12.2 Scatter plot of Monthly Rainfall

Aim: To develop a python program to create a scatter plot showing monthly rainfall data.

Algorithm:

1. Import matplotlib.
2. Create list for months and rainfall values.
3. Use the scatter() function.
4. Add title and label.
5. Display the graph.

Program:

```
import matplotlib.pyplot as plt.
```

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
```

```
rainfall = [40, 55, 70, 95, 120, 85]
```

```
plt.scatter(months, rainfall)
```

```
plt.title("Monthly Rainfall")
```

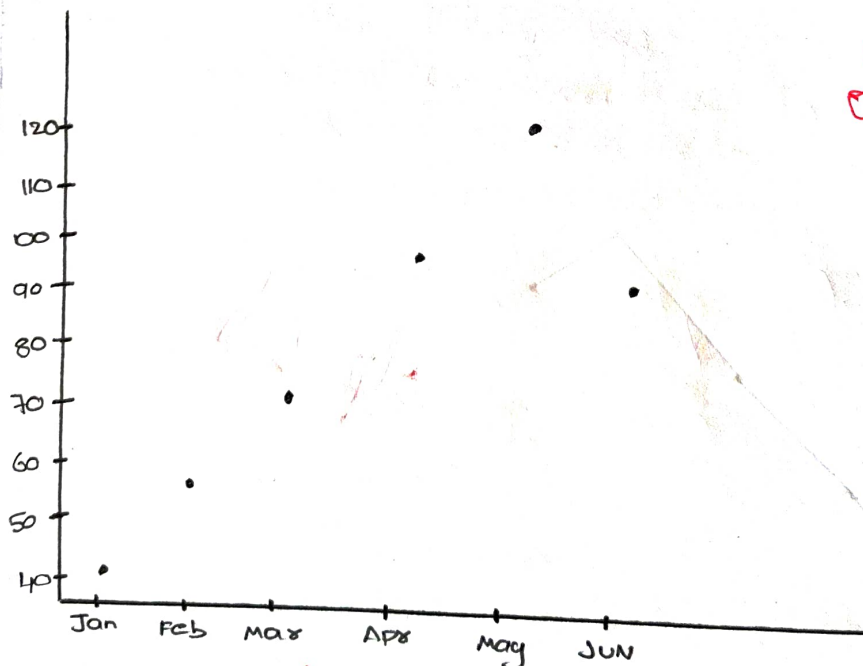
```
plt.xlabel("Months")
```

```
plt.ylabel("Rainfall (mm)")
```

```
plt.grid(True)
```

```
plt.show()
```

Output:



Result:

Thus, the python program to create a scatter plot for monthly rainfall was executed successfully.

13. Frequency Distribution of words in a text document.

Aim: To develop a python program that reads a text file and calculate the frequency distribution of words present in the document.

Algorithm :

1. Import the Counter class from the collection module.
2. Open the text file in read mode.
3. Read the contents of the file.
4. Convert all text to lowercase.
5. Split the text into individual words.
6. Count the frequency of each word using Counter.
7. Display the frequency of each word.

Program :

```
from collection import Counter
with open("Sample-text.txt", "r") as file :
    text = file.read().lower()
    words = text.split()
    frequency = Counter(words)
    print("word frequency distribution :")
    for word, count in frequency.items():
        print(word, ":" count)
```

Output :

```
word frequency distribution :
python : 3
is : 2
easy : 1
programming : 2
language : 1
```

slp ver

Result :

Thus, the python program to calculate the frequency distribution of words in a text document was executed successfully.

14. Frequency Distribution Customer Ages.

Aim: To develop a python using pandas to calculate the frequency distribution of customer ages.

Algorithm:

1. Import the pandas library.
2. Create a DataFrame containing customer age data.
3. Use the value_counts() function to calculate the frequency distribution.
4. Display the frequency of each age.

Program:

```
import pandas as pd
data = {
    'Age': [22, 25, 22, 30, 25, 35, 22, 30, 40, 25]
}
df = pd.DataFrame(data)
frequency = df['Age'].value_counts().sort_index()
print("Frequency Distribution of Customer Ages:")
print(frequency)
```

Output:

Frequency Distribution of Customer Ages :

22	2
25	3
30	2
35	1
40	1

Name: Age, dtype: int64

Result:

Thus the python program to calculate the frequency distribution of customer ages using pandas was executed successfully.

15 Frequency Distribution of Likes Among Social Media posts

Aim: To develop a python program that calculate the frequency distribution of likes received by social media posts.

Algorithm:

1. Import the pandas library
2. Create a dataframe containing the number of likes.
3. Use the value - count() function to count the occurrence of each like value.
4. Display the frequency distribution.

Program:

```
import pandas as pd
data = {
    'likes': [100, 120, 150, 100, 200, 150, 120, 100, 250, 200]
}
df = pd.DataFrame(data)
frequency = df['likes'].value_counts().sort_index()
print("Frequency Distribution of likes:")
print(frequency)
```

Output:

Frequency Distribution of likes:

100	3
120	2
150	2
200	2
250	1

Name: likes, dtype: int64

Result:

Thus the Python program to calculate the frequency distribution of like among social media posts was executed successfully.

16. word frequency distribution in Customer Reviews.

Aim: To develop a python program to calculate the frequency distribution of words in a set of customer reviews.

Algorithm:

1. Start the program
2. Store the customer reviews in a list.
3. Combine all the reviews into a single string
4. Convert the text into lowercase.
5. Remove punctuation from the text.
6. Split the text into individual words.
7. Count the frequency of the each word using counter.
8. Display the each word and its frequency.
9. Stop the program.

Code:

```
from collections import Counter
import re

reviews = [
    "This product is very good",
    "I really like this product",
    "This product is good and useful",
    "very useful product and good quality"
]

text = "".join(reviews)
text = text.lower()
text = re.sub(r'[\W|\s]', "", text)
words = text.split()
frequency = Counter(words)

print("word frequency distribution:")
for word, count in frequency.items():
    print(word, ":", count)
```

Output:

Word Frequency Distribution:

this : 2

product : 4

is : 2

very : 2

good : 3

i : 1

really : 1

like : 1

the : 1

and : 2

useful : 2

quality : 2

Result:

Thus, the python program was successfully implemented to calculate and display the frequency distribution of words in customer reviews.

17. Word Frequency Analysis from CSV

Aim: To develop a python program to load customer feedback from a csv file, preprocess the text, remove stop words, calculate word frequency, display the top N words, and visualize the results using a bar graph.

Algorithm:

1. Start the program
2. Import pandas, re, matplotlib, and Counter.
3. Load the data.csv file using pandas.
4. Read the feedback column from the dataset.
5. Combine all feedback into a single text.
6. Convert the text to lowercase.
7. Display the graphs.

Code:

```
import pandas as pd.  
import re  
import matplotlib.pyplot as plt.  
from collections import Counter.  
df = pd.read_csv("data.csv")  
text = " ".join(df["feedback"].astype(str))  
text = text.lower()  
text = re.sub(r'([^\w\s])', '', text)  
words = text.split()  
stop_words = {  
    "the", "is", "a", "an", "and", "or",  
    "this", "that", "of", "to", "in", "for",  
    "it", "very", "q"  
}  
words = [word for word in words if word not in stop_words]  
frequency = Counter(words)  
N = input("Enter the no. of top words to display: ")
```

top_words = frequency.most_common(N)

for word, count in top_words:

print(word, ":", count)

word_list = [item[0] for item in top_words]

counts = [item[1] for item in top_words]

plt.bar(word_list, counts)

plt.xlabel("words")

plt.ylabel("frequency")

plt.title("TOP N most frequent words")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

output:

Enter the no. of TOP words to display : 4

Top 4 most frequent words:

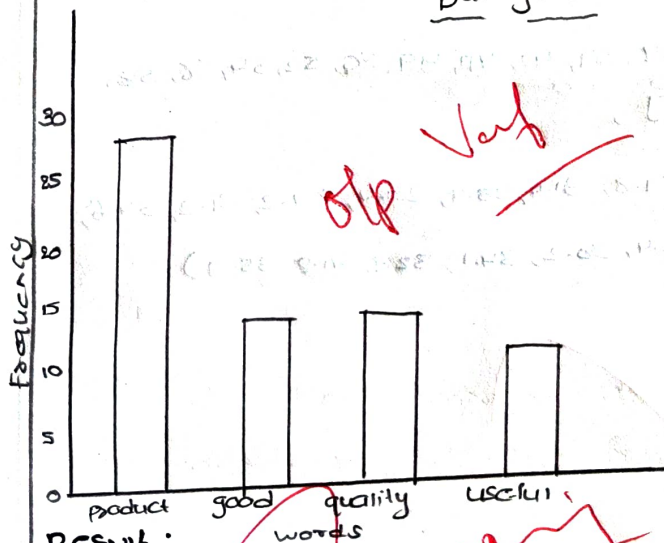
product : 29

good : 12

quality : 12

useful : 9

Bar graph



Thus, the python program executed successfully

18. Statistical Analysis of Age and Body Fat.

110

Aim: To calculate the mean, median and standard deviation of age and body fat percentage using pandas and to visualize the data using box plots, scatter plot, and q-plots.

Algorithm:

1. Start the program
2. Import pandas, Matplotlib and Scipy
3. Create a dataframe containing age and % Fat values.
4. Calculate the mean of age and % Fat.
5. Calculate the median of age and % Fat.
7. Draw box plots for age and % Fat.
8. Draw a scatter plot between age and % Fat.
9. Draw q-q plots to check the normality of both variables.
10. Display the results.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

data = {
    "Age": [23, 23, 27, 27, 29, 39, 41, 43, 49, 50, 52, 54, 56, 58,
            58, 60, 60, 62, 64],
    "% Fat": [9.5, 26.8, 7.8, 17.8, 31.4, 25.9, 27.4, 27.2, 31.2, 34.6,
              42.5, 28.8, 33.4, 30.2, 34.1, 32.9, 41.2, 35.7]
}
```

```
df = pd.DataFrame(data)
print("Mean:")
print(df.mean())
print("In Median:")
print(df.median())
print("In Standard Deviation")
print(df.std())
```

```
df = boxplot (column = ["Age", "% Fat"])
```

```
plt.title ("Box plot of Age and Body Fat")
```

```
plt.show()
```

```
plt.scatter (df ["Age"], df ["% Fat"])
```

```
plt.xlabel ("Age")
```

```
plt.ylabel ("% Body Fat (0-100)")
```

```
plt.title ("Age vs body Fat")
```

```
plt.show()
```

```
stats.probplot (df ["% Fat"], dist = "norm", plot = plt)
```

```
plt.title ("Q-Q plot of Body Fat")
```

```
plt.show()
```

```
stats.probplot (df ["% Fat"], dist = "norm", plot = plt)
```

```
plt.title ("Q-Q plot of Body Fat")
```

```
plt.show()
```

Output:

Mean:

Age 47.222222

% Fat 28.783333

dtype: float64

Median:

Age 51.0

% Fat 30.7

dtype: float64

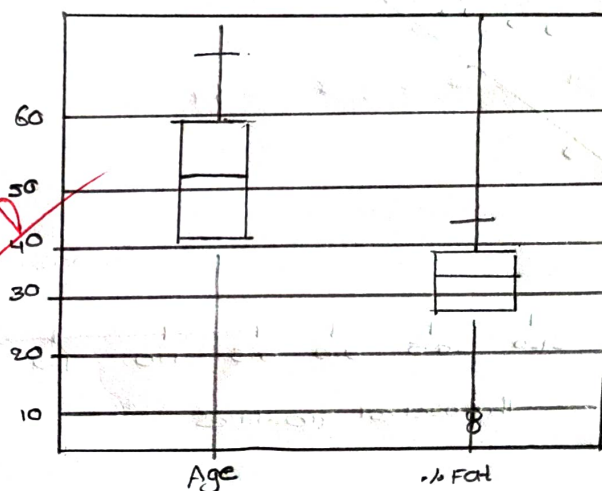
Standard Deviation:

Age 13.960261

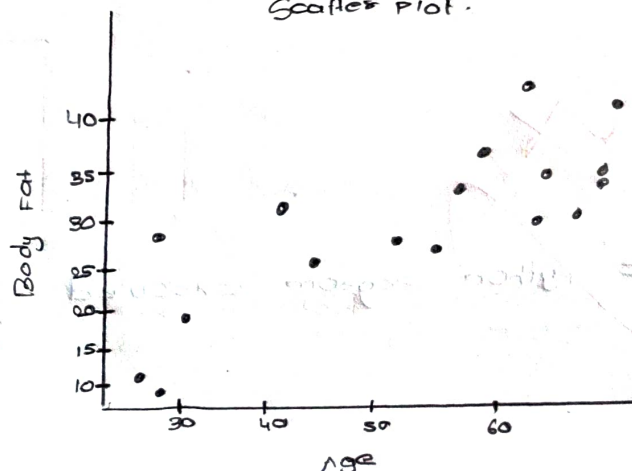
% Fat 9.254395

dtype: float64

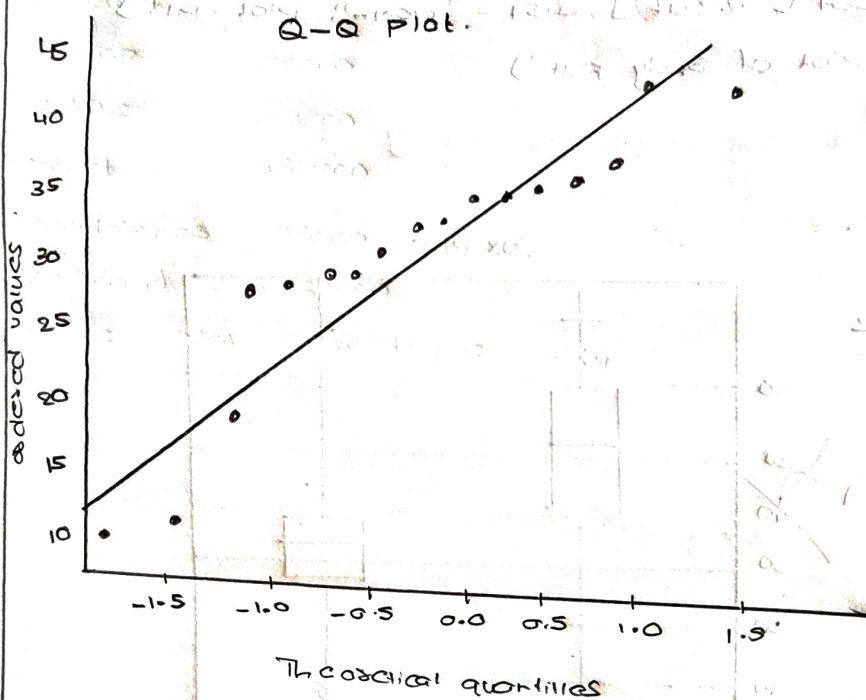
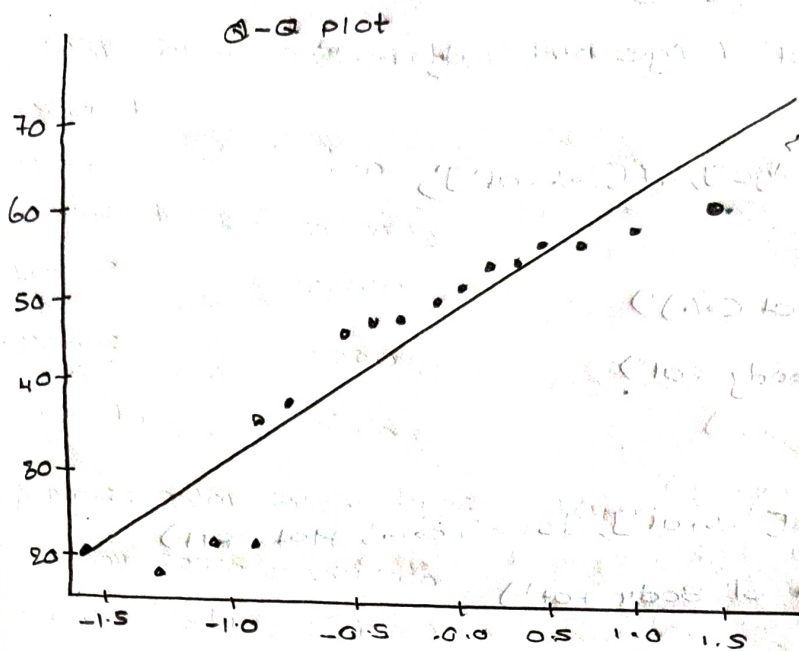
Box plot



Scatter plot.



Result:



Result: ~~XXX~~

Thus, the Python program executed successfully.

19. Sales and profit Analysis.

Output:

Aim: To load sales transactions data into a pandas database, calculate total sales for each transaction and product, determine the overall profit with a 20.1 profit margin, and display the top five products by total sales.

Algorithm:

1. Start the program
2. Import pandas
3. Load sales_data.csv
4. Read the date, product, quantity.
5. Calculate the total sales
6. Group the data by product.
7. Calculate total size for each product.
8. Calculate overall sales.
9. Display the top five products.

Code:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
df["Total Sales"] = df["Quantity Sold"] * df["Unit Price"]
product_sales = df.groupby("Product")["Total Sales"].sum()
overall_sales = df["Total Sales"].sum()
overall_profit = overall_sales * 0.20
print("Total sales for each product:")
print(product_sales)
print("\nOverall sales:", overall_sales)
print("Overall profit (20.1):", overall_profit)
tops = product_sales.sort_values(ascending=False).head(5)
print("\nTop 5 products:")
print(tops)
```

Output :

Total Sales for Each Product :

Product	
Headphones	75000
Keyboard	62000
Laptop	100000
Mobile	92500
Tablet	46000

Name : Total Sales, dtype : int64

Overall Sales : 2597000

Overall profit : 519400.0

Top 5 products :

Product	
Laptop	100000
Mobile	925000
Tablet	460000
Headphones	75000
Keyboard	57000

Name : Total Sales, dtype : int64

Result :

Thus, the python program executed successfully.

20. Customer Segmentation.

: 10/1/20

Aim: To load customer data into a Pandas DataFrame, Segment customers into High Spenders, Medium Spenders, and Low Spenders based on Spending, and calculate the average age of Customers in each segment.

Algorithm:

1. Start the program
2. Import pandas.
3. Load Customer_data.csv.
4. Read the customer ID, Age, Total Spending Columns.
5. Define Spending Categories.
6. otherwise, classify as low Spender
7. Add the segment as a new column.

Code:

```

import pandas as pd

df = pd.read_csv("customer_data.csv")

def segment(Spending):
    if Spending >= 1000:
        return "High Spenders"
    elif Spending >= 500:
        return "Medium Spenders"
    else:
        return "Low Spenders"

df["Spending segment"] = df["Total Spending"].apply(segment)

average_age = df.groupby("Spending segment")["Age"].mean()

print("Customer Segmentation:")
print(df)
print("\n Average Age of Each Spending Segment:")
print(average_age)

```

output :

Customer Segmentation

	customer ID	Age	Gender	Total spending	Spending segment
0	C001	22	MALE	350	Low
1	C002	35	FEMALE	750	Medium
2	C003	42	MALE	1250	High
3	C004	28	FEMALE	450	Low
4	C005	51	MALE	1500	Medium
5	C006	31	MALE	680	High
6	C007	45	FEMALE	1100	Low
7	C008	24	MALE	300	High

OK Ver

Result:

Thus, the python program executed successfully.